

Breadth-first search

Breadth-first search (**BFS**) is an algorithm for searching a tree data structure for a node that satisfies a given property. It starts at the tree root and explores all nodes at the present depth prior to moving on to the nodes at the next depth level. Extra memory, usually a queue, is needed to keep track of the child nodes that were encountered but not yet explored.

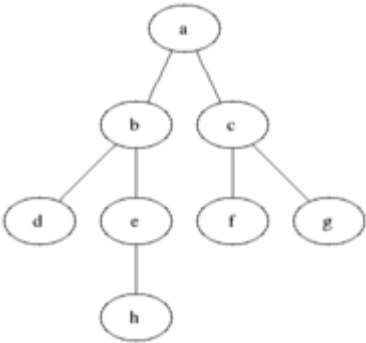
For example, in a chess endgame, a chess engine may build the game tree from the current position by applying all possible moves and use breadth-first search to find a winning position for White. Implicit trees (such as game trees or other problem-solving trees) may be of infinite size; breadth-first search is guaranteed to find a solution node^[1] if one exists.

In contrast, (plain) depth-first search (DFS), which explores the node branch as far as possible before backtracking and expanding other nodes,^[2] may get lost in an infinite branch and never make it to the solution node. Iterative deepening depth-first search avoids the latter drawback at the price of exploring the tree's top parts over and over again. On the other hand, both depth-first algorithms typically require far less extra memory than breadth-first search.^[3]

Breadth-first search can be generalized to both undirected graphs and directed graphs with a given start node (sometimes referred to as a 'search key').^[4] In state space search in artificial intelligence, repeated searches of vertices are often allowed, while in theoretical analysis of algorithms based on breadth-first search, precautions are typically taken to prevent repetitions.

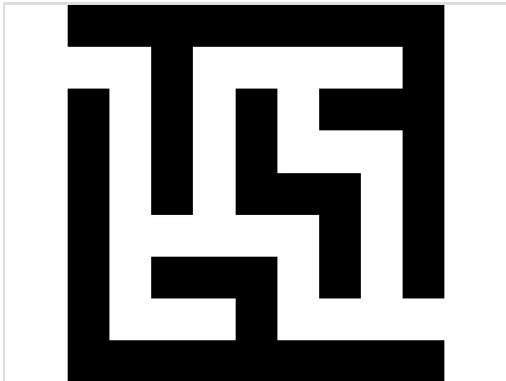
BFS and its application in finding connected components of graphs were invented in 1945 by Konrad Zuse, in his (rejected) Ph.D. thesis on the Plankalkül programming language, but this was not published until 1972.^[5] It was reinvented in 1959 by Edward F. Moore, who used it to find the shortest path out of a maze,^{[6][7]} and later developed by C. Y. Lee into a wire routing algorithm (published in 1961).^[8]

Breadth-first search



Animated example of a breadth-first search.
Black = explored, grey = queued to be explored later on

Class	<u>Search algorithm</u>
Data structure	<u>Graph</u>
<u>Worst-case performance</u>	$O(V + E)$
<u>Worst-case space complexity</u>	$O(V)$
Optimal	Yes (always finds shortest paths)



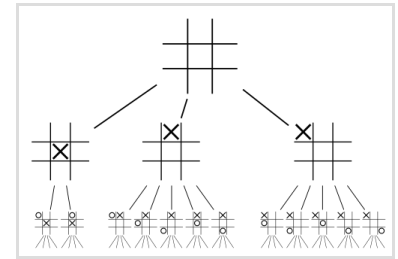
BFS on Maze-solving algorithm

Pseudocode

The below pseudocode finds the shortest path from a given *root* vertex to all other vertices in the graph using BFS.

Input: A graph G and a starting vertex *root* of G

Output: Goal state. The *parent* links trace the shortest path back to *root*^[9]



Top part of Tic-tac-toe game tree

```

1  procedure BFS( $G$ ,  $root$ ) is
2      let  $Q$  be a queue
3      label  $root$  as explored
4       $Q.enqueue(root)$ 
5      while  $Q$  is not empty do
6           $v := Q.dequeue()$ 
7          if  $v$  is the goal then
8              return  $v$ 
9          for all edges from  $v$  to  $w$  in  $G.adjacentEdges(v)$  do
10             if  $w$  is not labeled as explored then
11                 label  $w$  as explored
12                  $w.parent := v$ 
13                  $Q.enqueue(w)$ 

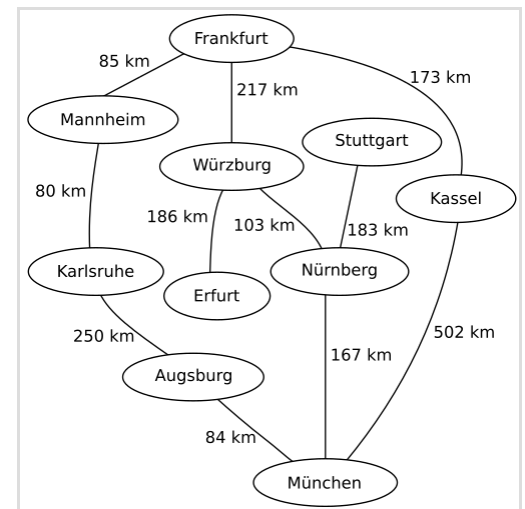
```

More details

This non-recursive implementation is similar to the non-recursive implementation of depth-first search, but differs from it in two ways:

1. it uses a queue (First In First Out) instead of a stack (Last In First Out) and
2. it checks whether a vertex has been explored before enqueueing the vertex rather than delaying this check until the vertex is dequeued from the queue.

If G is a tree, replacing the queue of this breadth-first search algorithm with a stack will yield a depth-first search algorithm. For general graphs, replacing the stack of the iterative depth-first search implementation with a queue would also produce a breadth-first search algorithm, although a somewhat nonstandard one.^[10]



An example map of Southern Germany with some connections between cities

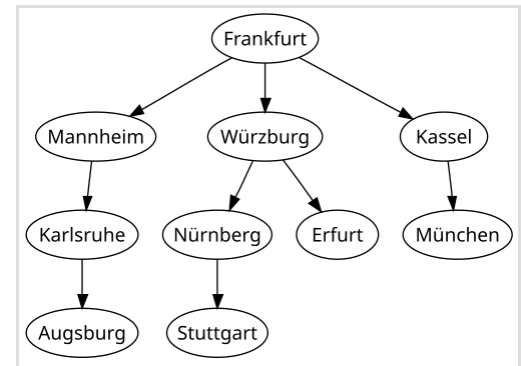
The queue Q contains the frontier along which the algorithm is currently searching.

Nodes can be labelled as explored by storing them in a set, or by an attribute on each node, depending on the implementation.

The *parent* attribute of each node is useful for accessing the nodes in a shortest path, for example

by backtracking from the destination node up to the starting node, once the BFS has been run, and the predecessors nodes have been set.

Breadth-first search produces a *breadth-first tree*. The diagrams on the right show the breadth-first tree obtained by running a BFS on an example graph of German cities (upper diagram) starting from *Frankfurt*.



The breadth-first tree obtained when running BFS on the given map and starting in Frankfurt

Analysis

Time and space complexity

The time complexity can be expressed as $O(|V| + |E|)$, as every vertex and every edge will be explored in the worst case. $|V|$ is the number of vertices and $|E|$ is the number of edges in the graph. Note that $O(|E|)$ may vary between $O(1)$ and $O(|V|^2)$, depending on how sparse the input graph is.^[11]

When the number of vertices in the graph is known ahead of time, and additional data structures are used to determine which vertices have already been added to the queue, the space complexity can be expressed as $O(|V|)$, where $|V|$ is the number of vertices. This is in addition to the space required for the graph itself, which may vary depending on the graph representation used by an implementation of the algorithm.

When working with graphs that are too large to store explicitly (or infinite), it is more practical to describe the complexity of breadth-first search in different terms: to find the nodes that are at distance d from the start node (measured in number of edge traversals), BFS takes $O(b^{d+1})$ time and memory, where b is the "branching factor" of the graph (the average out-degree).^{[12]:81}

Completeness

In the analysis of algorithms, the input to breadth-first search is assumed to be a finite graph, represented as an adjacency list, adjacency matrix, or similar representation. However, in the application of graph traversal methods in artificial intelligence the input may be an implicit representation of an infinite graph. In this context, a search method is described as being complete if it is guaranteed to find a goal state if one exists. Breadth-first search is complete, but depth-first search is not. When applied to infinite graphs represented implicitly, breadth-first search will eventually find the goal state, but depth first search may get lost in parts of the graph that have no goal state and never return.^[13]

BFS ordering

An enumeration of the vertices of a graph is said to be a BFS ordering if it is a possible output of the application of BFS to this graph.

Let $G = (V, E)$ be a graph with n vertices. Recall that $N(v)$ is the set of neighbors of v . Let $\sigma = (v_1, \dots, v_m)$ be a list of distinct elements of V , for $v \in V \setminus \{v_1, \dots, v_m\}$, let $\nu_\sigma(v)$ be the least i such that v_i is a neighbor of v , if such a i exists, and be ∞ otherwise.

Let $\sigma = (v_1, \dots, v_n)$ be an enumeration of the vertices of V . The enumeration σ is said to be a BFS ordering (with source v_1) if, for all $1 < i \leq n$, v_i is the vertex $w \in V \setminus \{v_1, \dots, v_{i-1}\}$ such that $\nu_{(v_1, \dots, v_{i-1})}(w)$ is minimal. Equivalently, σ is a BFS ordering if, for all $1 \leq i < j < k \leq n$ with $v_i \in N(v_k) \setminus N(v_j)$, there exists a neighbor v_m of v_j such that $m < i$.

Applications

Breadth-first search can be used to solve many problems in graph theory, for example:

- Copying garbage collection, [Cheney's algorithm](#)
- Finding the [shortest path](#) between two nodes u and v , with path length measured by number of edges (an advantage over [depth-first search](#))^[14]
- [\(Reverse\) Cuthill–McKee](#) mesh numbering
- [Ford–Fulkerson method](#) for computing the [maximum flow](#) in a [flow network](#)
- Serialization/Deserialization of a binary tree vs serialization in sorted order, allows the tree to be re-constructed in an efficient manner.
- Construction of the *failure function* of the [Aho–Corasick](#) pattern matcher.
- Testing [bipartiteness](#) of a graph.^[15]
- Implementing parallel algorithms for computing a graph's transitive closure.^[16]

See also

- [Depth-first search](#) – Algorithm to search the nodes of a graph
- [Dijkstra's algorithm](#) – Algorithm for finding shortest paths
- [Iterative deepening depth-first search](#) – Tree searching strategy
- [Level structure](#) – Object in graph theory
- [Lexicographic breadth-first search](#) – Partition-based graph traversal method
- [Parallel breadth-first search](#) – Parallel version of breadth-first search algorithm

References

1. that is, a node satisfying the specified property
2. Cormen Thomas H.; et al. (2009). "22.3". *Introduction to Algorithms*. MIT Press.

3. Korf, Richard E. (1985). "Depth-First Iterative Deepening: An Optimal Admissible Tree Search" (<https://doi.org/10.7916/D8HQ46X1>). *Artificial Intelligence* (27): 99–100. doi:10.7916/D8HQ46X1 (<https://doi.org/10.7916%2FD8HQ46X1>).
4. "Graph500 benchmark specification (supercomputer performance evaluation)" (<https://web.archive.org/web/20150326055019/http://www.graph500.org/specifications#sec-5#sec-5>). Graph500.org, 2010. Archived from the original (<http://www.graph500.org/specifications#sec-5>) on 2015-03-26. Retrieved 2015-03-15.
5. Zuse, Konrad (1972), *Der Plankalkül* (<http://zuse.zib.de/item/gHI1cNsUuQweHB6>) (in German), Konrad Zuse Internet Archive. See pp. 96–105 of the linked pdf file (internal numbering 2.47–2.56).
6. Moore, Edward F. (1959). "The shortest path through a maze". *Proceedings of the International Symposium on the Theory of Switching*. Harvard University Press. pp. 285–292. As cited by Cormen, Leiserson, Rivest, and Stein.
7. Skiena, Steven (2008). "Sorting and Searching". *The Algorithm Design Manual*. Springer. p. 480. Bibcode:2008adm..book.....S (<https://ui.adsabs.harvard.edu/abs/2008adm..book.....S>). doi:10.1007/978-1-84800-070-4_4 (https://doi.org/10.1007%2F978-1-84800-070-4_4). ISBN 978-1-84800-069-8.
8. Lee, C. Y. (1961). "An Algorithm for Path Connections and Its Applications". *IRE Transactions on Electronic Computers* (3): 346–365. doi:10.1109/TEC.1961.5219222 (<https://doi.org/10.1109%2FTEC.1961.5219222>). S2CID 40700386 (<https://api.semanticscholar.org/CorpusID:40700386>).
9. Cormen, Thomas H. (January 2010). "22.2 Breadth-first search". *Introduction to algorithms* (<http://worldcat.org/oclc/1006880283>). Prentice-Hall Of India Pvt. Limited. ISBN 978-81-203-4007-7. OCLC 1006880283 (<https://search.worldcat.org/oclc/1006880283>).
10. "Stack-based graph traversal ≠ depth first search" (<https://11011110.github.io/blog/2013/12/17/stack-based-graph-traversal.html>). *11011110.github.io*. Retrieved 2020-06-10.
11. Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001) [1990]. "22.2 Breadth-first search". *Introduction to Algorithms* (2nd ed.). MIT Press and McGraw-Hill. pp. 531–539. ISBN 0-262-03293-7.
12. Russell, Stuart; Norvig, Peter (2003) [1995]. *Artificial Intelligence: A Modern Approach* (2nd ed.). Prentice Hall. ISBN 978-0137903955.
13. Coppin, B. (2004). *Artificial intelligence illuminated*. Jones & Bartlett Learning. pp. 79–80.
14. Aziz, Adnan; Prakash, Amit (2010). "4. Algorithms on Graphs". *Algorithms for Interviews*. Algorithmsforinterviews.com. p. 144. ISBN 978-1453792995.
15. Kleinberg, Jon; Tardos, Éva (2006), *Algorithm Design*, Addison Wesley, pp. 94–97.
16. Dhulipala, Laxman; Blelloch, Guy E.; Shun, Julian (August 21, 2019). *Theoretically Efficient Parallel Graph Algorithms Can Be Fast and Scalable*. p. 17. arXiv:1805.05208 (<https://arxiv.org/abs/1805.05208>). doi:10.1145/3210377.3210414 (<https://doi.org/10.1145%2F3210377.3210414>). ISBN 9781450357999. S2CID 44126609 (<https://api.semanticscholar.org/CorpusID:44126609>).

- Knuth, Donald E. (1997), *The Art of Computer Programming Vol 1. 3rd ed.* (<https://web.archive.org/web/20080904163709/http://www-cs-faculty.stanford.edu/~knuth/taocp.html>), Boston: Addison-Wesley, ISBN 978-0-201-89683-1, archived from the original (<http://www-cs-faculty.stanford.edu/~knuth/taocp.html>) on 2008-09-04, retrieved 2008-02-09

External links

- Open Data Structures - Section 12.3.1 - Breadth-First Search (https://web.archive.org/web/20141029100806/http://opendatastructures.org/versions/edition-0.1e/ods-java/12_3_Graph_Traversal.html#SECTION00153100000000000000), Pat Morin
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Breadth-first_search&oldid=1354179802"